

Arrays and Classes

Produced
by:

Dr. Siobhán Drohan,
Ms. Mairead Meagher,
Ms. Siobhán Roche.

Topics list

1. Arrays of different types

2. Arrays of primitive types

3. Arrays of objects

- String

- Person

Let's Look at arrays of different types

Arrays can store any type of data

Let's look at some examples:

1. Array of primitives - **int**
2. Array of objects – **String**
3. Array of objects - **Product**

An array can store any type of data.

Primitive Types

```
int numbers[] = new int[10];
```

```
byte smallNumbers[] = new byte[4];
```

```
char characters[] = new char[26];
```

Object Types

```
String[] words = new String[4];
```

```
Person[] friends = new Person[4];
```

Topics list

1. Arrays of different types

2. Arrays of primitive types

3. Arrays of objects

- String

- Person

Structure of an **int** primitive array

int[] numbers;

numbers

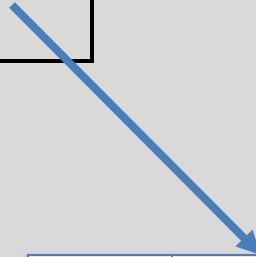
null

Structure of an **int** primitive array

```
int[] numbers;
```

```
numbers = new int[4];
```

numbers



0	0
1	0
2	0
3	0

Structure of an **int** primitive array

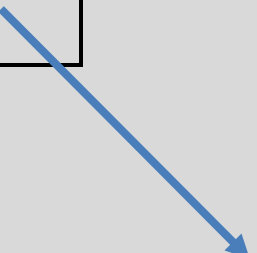
```
int[] numbers;
```

```
numbers = new int[4];
```

```
numbers[2] = 18;
```

We are directly
accessing the
element at index **2**
and setting it to a
value of **18**.

numbers



0	0
1	0
2	18
3	0

Structure of an **int** primitive array

```
int[] numbers;
```

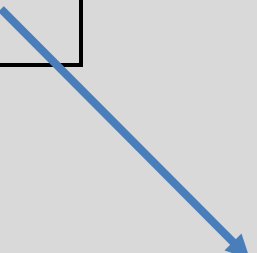
```
numbers = new int[4];
```

```
numbers[2] = 18;
```

```
numbers[0] = 12;
```

We are setting the
element at index **0**
to a value of **12**.

numbers



0	12
1	0
2	18
3	0

Structure of an **int** primitive array

```
int[] numbers;
```

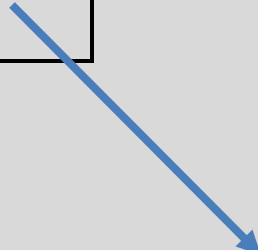
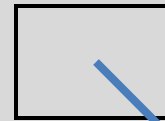
```
numbers = new int[4];
```

```
numbers[2] = 18;
```

```
numbers[0] = 12;
```

```
print(numbers[2]);
```

numbers



0	12
1	0
2	18
3	0

Here we are printing the contents of
index location 2
i.e. 18 will be printed to the console.

Topics list

1. Arrays of different types
2. Arrays of primitive types

3. Arrays of objects

- String 
- Person

An array can store any type of data.

Primitive Types

```
int numbers[] = new int[10];
```

```
byte smallNumbers[] = new byte[4];
```

```
char characters[] = new char[26];
```

Object Types

```
String[] words = new String[4];
```

```
Person[] friends = new Person[4];
```



Structure of a **String** object array

String[] words;

words

null

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

words



0	null
1	null
2	null
3	null

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

```
words[1] = "Dog";
```

words



0	null
1	
2	null
3	null



"Dog"

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

```
words[1] = "Dog";
```

We are directly accessing the element at index **1** and setting it to a value of **"Dog"**.

words



0	null
1	
2	null
3	null



"Dog"

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

```
words[1] = "Dog";
```

```
words[3] = "Cat";
```

words



0	null
1	
2	null
3	



"Dog"



"Cat"

Structure of a **String** object array

```
String[] words;
```

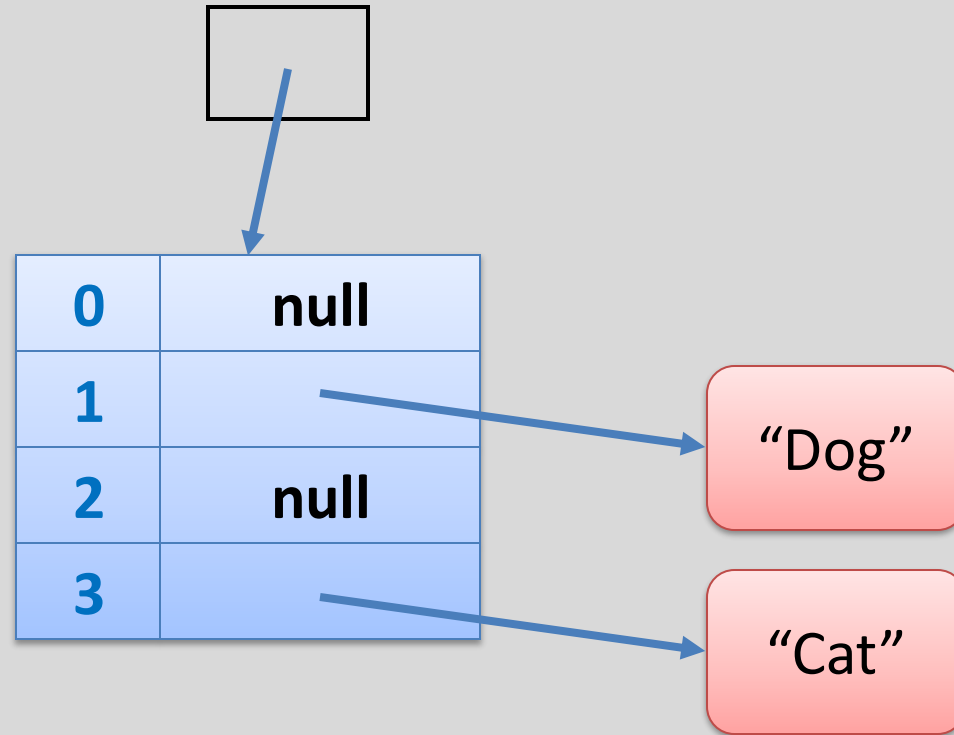
```
words = new String[4];
```

```
words[1] = "Dog";
```

```
words[3] = "Cat";
```

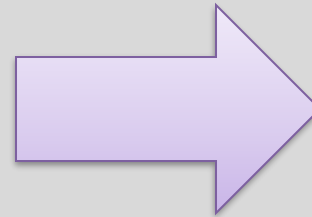
The element at index
3 is set to **"Cat"**.

words



Structure of a **String** object array

```
String words[];  
  
words = new String[4];  
  
words[1] = "Dog";  
words[3] = "Cat";  
  
for (int i=0; i < words.length; i++)  
{  
    System.out.println(words[i]);  
}
```



```
null  
Dog  
null  
Cat
```

Topics list

1. Arrays of different types
2. Arrays of primitive types

3. Arrays of objects

- String

- Person



An array can store any type of data.

Primitive Types

```
int numbers[] = new int[10];
```

```
byte smallNumbers[] = new byte[4];
```

```
char characters[] = new char[26];
```

Object Types

```
String[] words = new String[4];
```

```
Person[] friends = new Person[4];
```



Person Class

**Object Type/
Class Name**

Fields

i.e. the attributes of
the class

Methods

i.e. the behaviours of
the class

Person Class Diagram

 Person

- int age
- String firstName
- String secondName
- float temperature

- Person(String firstName, String secondName, int age)

- int getAge()
- String getFirstName()
- String getSecondName()
- float getTemperature()

- void setAge(int age)
- void setFirstName(String firstName)
- void setSecondName(String firstName)
- void setTemperature(float temperature)

- void printAge()
- void printFirstName()
- void printSecondName()
- void printFullName()
- void printTemperature()

- int happyBirthday()

Structure of a primitive array of **Person**

Person[] friends;

friends

null

Structure of a **Product** primitive array

Person[] friends;

friends = new Person[4];

friends



0	null
1	null
2	null
3	null

Structure of a **Product** primitive array

```
Person[] friends;
```

```
friends = new Person[4];
```

products



0	null
1	
2	null
3	null

private int age	25
private String firstName	"Joey"
private String secondName	"Tribbiani"
private float temperature	13.2

```
friends[1] = new Person("Joey", "Tribbiani", 25,);
```

Example using a **Person** object array

```
public void printFriends() {  
    if (size == 0 ) return "Sorry no Friends";  
  
    else {  
        for (int i = 0; i < friends.length; i++) {  
            friends[i]. printFirstName();  
            friends[i]. printSecondName();  
        }  
    }  
}
```

Prints first and second names of all friends stored in the primitive array.

Example using a **Person** object array

```
public String listOfFriends(){  
  
    String listOfStudents = "List of friends \n";  
  
    for (int i = 0; i < friends.length; i++) {  
  
        listOfStudents += friends[i].getFirstName() + " "  
                        + friends[i].getSecondName() + "\n";  
  
        return listOfStudents;  
    }  
}
```

Returns the first and second names of all friends stored in the primitive array.

Questions?

